Proceedings of 2024 IEEE 12th International Conference on Intelligent Systems (IS)

RAGSQL: context retrieval evaluation on augmenting text-to-SQL prompts

Sergey Vichev
Sofia University "St. Kliment Ohridski"
Faculty of Economics and Business Administration
Department of Statistics and Econometrics
Email: sergey@crisplabs.ai

Angel Marchev, Jr.
University of National and World Economy
Faculty of Management and Administration
Department of Management
Email: angel.marchev@unwe.bg

***Abstract*—Text-to-SQL area evolves rapidly in the era of large language models (LLMs) as organizations look for adapting the technology. A gained popularity method to improve LLM outputs is through giving relevant context about the database and organizational data as few-shot prompting. Recent research showcases that better results in text-to-SQL tasks can be achieved in a Retrieval Augmented Generation (RAG) setting by augmenting the prompt context in dynamic fashion during the LLM inference. In this paper we design such experiment by adapting data from BIRD benchmark, on more than 10 thousand examples and evaluate existing embedding models, both closed and open source, for such tasks reporting the comparison results. We then fine-tune an open-source sentence transformer model that outperforms the state-of-the-art (SOTA) embedding models on both question-to-evidence and question-to-SQL retrieval experiments.**

***Keywords*—text-to-sql; information retrieval; fine-tuning; sentence-transformers; Large Language Models**

I. Introduction

The advent of large language models (LLMs) has catalyzed a resurgence in the popularity of text-to-SQL generation area, a technology that empowers users to interact with relational databases through natural language queries. This capability is particularly valuable in business and research environments where data-driven decision-making is paramount but expertise in SQL is limited. The integration of LLMs into this process through strategies like prompt engineering[2], chain-of-thought reasoning[3], and retrieval-augmented generation (RAG) has introduced innovative ways to enhance the interpretability and accuracy of generated SQL queries.

Despite these advancements, the text-to-SQL generation process faces significant challenges that impede its efficacy and adoption. One of the existing hurdles is the management of input size of prompts that are being passed to LLMs, as often domain or database specific information is too large to being passed into a single prompt. Although techniques such as few-shot prompting and contextual learning aim to improve the model's understanding of the database schema and query intention, they are constrained by the prompt size limitations of current LLMs. As the complexity and length of inputs increase, the performance often degrades—a phenomenon likened to finding a 'needle in a haystack', where the critical information is obscured by volume. Moreover, large prompts lead to higher latency which is often critical for business applications. The integration of LLMs into the text-to-SQL generation process has introduced innovative ways to enhance the interpretability and accuracy of generated SQL queries. However, this process still faces significant challenges, particularly in managing the input size of prompts, which often contain extensive domain-specific information that exceeds the current prompt size limitations of LLMs [10],[13].

While RAG offers a promising approach by dynamically retrieving relevant information based on the user's query, thereby reducing the burden on the prompt itself, it introduces new complexities. For instance, the effectiveness of RAG depends heavily on the relevance of the information retrieved, which in turn hinges on the quality of the underlying similarity search algorithms. While there are benchmarks for various similarity searches [1], the specific application of information retrieval on text-to-SQL data remains under-explored. There is a noticeable gap in the evaluation frameworks specifically tailored to assess the impact of similarity searches in retrieving pertinent information that accurately informs SQL query generation.

TABLE I
TEXT-TO-SQL DATA

Question:	Who is the CBSA officer of the post point in the area with the highest number of employees?
Evidence:	CBSA officer refers to CBSA_name; the highest number of employees refers to MAX(employees)
SQL query	SELECT T1.CBSA_name FROM CBSA AS T1 INNER JOIN zip_data AS T2 ON T1.CBSA = T2.CBSA WHERE T2.employees = (SELECT MAX(employees) FROM zip_data)

Legend: red color denotes persona or entity tokens; blue - comparison or aggregation; yellow - geography

Moreover, as generally available embedding models are proficient at handling a broad spectrum of tasks, the specifics of text-to-SQL domain, such as understanding and mapping natural language queries to precise SQL code examples or evidences demand a more targeted approach in information

979-8-3503-5098-2/24/$31.00 ©2024 IEEE

retrieval. The syntactic and semantic nuances inherent in SQL queries necessitate adaptations that align closely with database schema and query intent, distinct from generic information retrieval tasks (Table I). Consequently, domain adaptation for these models in the context of information retrieval for SQL texts is beneficial to optimize performance of similarity search for text-to-SQL data.

Addressing these challenges could contribute to the research of text-to-SQL area leading to making data more accessible to non-expert users and enhancing the decision-making processes across various sectors.

Therefore, this paper aims to explore and benchmark different models used for similarity search and information retrieval specifically in the context of text-to-SQL We implement a custom evaluation framework and adapt the BIRD bench dataset making it suitable for information retrieval setting. Besides the existing models evaluation we fine-tune a pre-trained sentence transformer model and report the comparison results.

By evaluating information retrieval performance for text-to-SQL, we seek to establish a more robust framework that not only improves the accuracy of SQL generation but also expands the practical utility of natural language interfaces for complex database systems. This research could pave the way for more intuitive and efficient data querying tools, ultimately democratizing data access and analytics.

II. Literature Review

Several recent studies relevant to this study have enhanced the capabilities of text-to-SQL models through various innovative approaches. The recent advancements in text-to-SQL frameworks and RAG highlight the ongoing evolution in the field of NLP, particularly in how databases are queried using natural language. Despite the progress, the gap in achieving human parity emphasizes the need for continued research, particularly in domain adaptation and the management of long context dependencies in LLMs. The integration of sophisticated retrieval mechanisms and fine-tuning approaches such as those seen in Dubo-SQL may hold the key to bridging these gaps.

A. Emerging challenges in front of text-to SQL tasks

Foremost challenge is the so called needle-in-the-haystack (or large-prompt) problem. Liu [7] explores challenges large language models (LLMs) face when dealing with extensive prompts. These challenges are pertinent as LLMs, like those used in text-to-SQL tasks, often require processing long and complex queries, which can lead to degradation in performance due to the dilution of context as the prompt length increases [14]. This observation sets a foundational understanding of the limitations present in current LLMs when applied in tasks requiring deep contextual understanding over large text spans. Next are the domain adaptation challenges as reviewed in [8]. The paper discusses the hurdles encountered when adapting advanced text-to-SQL models to specific domains. It highlights the gaps in model performance when transferred to new domains, underscoring the need for robust domain adaptation strategies to maintain efficacy across varied data environments.

B. Evolution of Retrieval-Augmented Generation

Retrieval-Augmented Generation [10] has become a cornerstone technique in improving the performance of NLP tasks by combining the power of retrieval from large databases with generative capabilities. This approach has proven particularly effective in text-to-SQL tasks, where additional context from related texts significantly enhances the quality of the generated SQL queries [15].

In that regard [5] introduces a framework that utilizes RAG to enhance text-to-SQL translation. This model dynamically revises SQL queries based on a repository of sample SQL queries and fine-tuned feedback mechanisms, showing improved accuracy by leveraging past query examples to inform current query generation. Li [4] presents an advanced model that incorporates diverse retrieval techniques and fine-tuning mechanisms to improve the performance of text-to-SQL generation. By integrating multiple sources and methods for context retrieval and adaptation, this model represents a significant step forward in creating more flexible and accurate text-to-SQL translation tools.

C. Some methodological tools used in the current study

The utilization of Sentence-BERT, as explored in [12] has provided a method for generating meaningful and contextually rich sentence embeddings. These embeddings are crucial for tasks that require dynamic information retrieval, accurate matching of natural language queries to relevant data is essential [17].

BIRD Benchmark Analysis [9] has become a staple for evaluation of the performance of LLMs, concluding that while promising, LLMs still lag behind human performance in generating accurate SQL queries from natural language questions. This suggests that current models still require significant enhancements to match human-like efficiency and accuracy in database querying tasks.

III. Methodology

This study leverages the first phase of RAG focused on enhancing text-to-SQL conversions through evidence-based prompting. Our methodology is specifically tailored for handling unlabeled question-pairs, which involves no initial ranking of the provided data.

A. Data Preparation and Initial Retrieval

We initiate our methodology by extracting potential negative pairs from a comprehensive set of evidence, not limited strictly to relevant databases. The cosine similarity for all potential combinations of question-evidence pairs is computed and stored in tensors, facilitating a comprehensive search space beyond conventional retrieval boundaries.

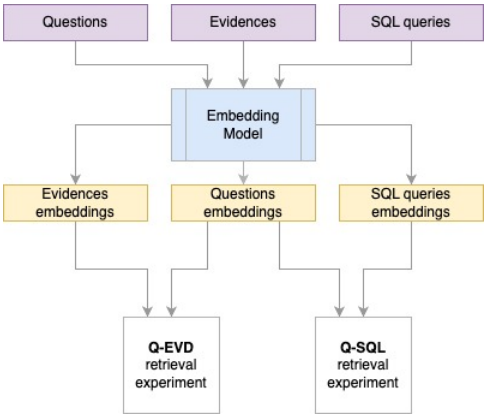


Fig. 1. Method diagram

Algorithm 1 Evaluator Class - Methodology for Computing Cosine Similarities and Evaluation Metrics

procedure COSINE_SIMILARITY_BATCH
    Input: query and candidate embeddings
    Process:
    Convert embeddings to tensors
    Compute dot product and norms for cosine similarity
    Output: Matrix of cosine similarities
end procedure
procedure GET_TOP_K_LABELS
    Input: Cosine similarity matrix
    Process:
    Sort and retrieve top-k indices and labels
    Output: List of top-k labels for each query
end procedure
procedure CALCULATE_METRICS
    Input: top_k_labels, context_labels, subset_k, metrics
    Process:
    Calculate hits and NDCG for the top k labels
    Output: hits and ndcg values
end procedure

Let X and Y be matrices representing the embeddings, where the rows correspond to different embeddings and the columns to the features of these embeddings.

1. Compute the dot product of X and the transpose of Y:

$$S = XY^T$$

2. Compute the norms: We compute the norm of a tensor using the general p-norm formula, which is defined as:

$$\|x\|_p = \left(\sum_{i=1}^{n} |x_i|^p \right)^{\frac{1}{p}}$$

where x is a tensor, x_i denotes the i-th element of x, n is the total number of elements in the tensor, and p is the norm order. The most commonly used norm, the Euclidean norm (or 2-norm), is computed as:

$$\|x\|_2 = \sqrt{\sum_{i=1}^{n} x_i^2}$$

3. Compute the cosine similarities as a matrix C:

$$C = \frac{S}{N_X N_Y}$$

Here, the division is element-wise between S and the outer product of N_X and N_Y, which gives the cosine similarity between each pair of vectors (one from X and one from Y).

B. Sorting and Selection

For each question, the computed similarities are sorted in descending order. This arrangement allows us to select the top candidates up to a maximum of K (max$K$), thus prioritizing pairs that are most likely to yield valuable insights upon deeper analysis.

Mathematically, $argsort$ represents an indirect sort i.e. sort an array or a data sequence's indices based on the sort order of the values in the array.

Given an array S of n elements, the operation returns an array I of the same shape as S containing the indices that would sort S in descending order. If S is sorted into a new array $\dot{S}$ such that each element $\dot{S}[i]$ is $S[I[i]]$, then I is the output of $argsort$. This operation doesn't sort the array itself but provides the indices that can be used to produce a sorted array.

4. For a given array $S = [a_1, a_2, ..., a_n]$, the index array I after executing $I =$ argsort(A) is such that $A[I[0]], A[I[1]], ..., A[I[n-1]]$ are in sorted in descending order.

5. Having the matrix S where each row S_i contains numerical values that are sorted in descending order, the top k elements of each row can be selected after sorting the indices. Mathematically, this is represented as:

$$T = [S_{i,1}, S_{i,2}, \ldots, S_{i,k}] \quad \text{for } i = 1, \ldots, n$$

C. Relevance Scoring

Within the selected max$K$ indices, the algorithm calculates the Hit Rate the Normalized Discounted Cumulative Gain (NDCG) for each question and K. This process involves checking whether the index of the question aligns with the index of the evidence and thus for each question HitRate@K and NDCG@K is calculated.

6. Hits Calculation: Define a vector $\mathbf{hits}$ where each element $\mathbf{hits}_i$ is calculated as:

$$H_i = I(L_i \in T)$$

where:

- I(condition) is 1 if the condition is true, and 0 otherwise.
- L is an array of context labels

7. NDCG Calculation:

$$\text{NDCG}_i = \begin{cases} \frac{1}{\log_2(P[L_i]+1)} & L_i \in P \\ 0 \end{cases} \text{ for } i = 1, \ldots, n$$

where P is a dictionary of label positions for the top k labels in T_i, where each label is mapped to its rank starting from 1

D. Data Categorization

The output generated for each question, termed as raw data, undergoes a categorization process based on predefined criteria such as model, train/test subsets, and database specifics. This step ensures that subsequent analyses are structured and contextualized according to the intrinsic characteristics of the data.

E. Evaluation Metrics

We employ Precision@K and NDCG@K, where K is 3, 5, or 10 retrieved elements to assess the efficacy of the selected embedding models. This metric comparison not only highlights the performance of each model but also are widely accepted and comparable with established retrieval benchmarks within the RAG framework.

Algorithm 2 Evaluation and Metrics Calculation in Text-to-SQL Context

```
procedure EVALUATE
    Input: DataFrame with queries and metadata
    Process:
    Iterate and query collection for each record
    Check hits by comparing IDs
    Output: Lists of results, result IDs, and hits
end procedure
procedure CALCULATE_METRICS
    Input: data, k_values, metrics
    Process:
    Compute mean metrics grouped by label
    Output: DataFrame with calculated means
end procedure
```

This methodology aims to robustly test the efficacy of the embedding models on RAG framework in enhancing text-to-SQL conversions by systematically selecting and evaluating relevant context from unlabeled question-pairs.

IV. Experimental design

A. Dataset adaptation

For experiments we adapt the dataset from the widely accepted cross-domain BIRD benchmark designed for large scale text-to-SQL evaluation and covering more than 37 professional domains, 12,751 unique question-SQL pairs and 95 databases[4]. An example of a record is provided in Table I.

In our experimental setting the main task is finding the relevant evidence or SQL query given the question. For each question there is only one correct evidence or SQL query, which we assume to be the one provided by BIRD dataset. Thus the dataset contains N correct question-evidence or question-SQL query pairs and N-1 incorrect evidences or SQL queries for any give question.

BIRD benchmark dataset originally is spitted into training, development and test sets. While we don't have any access to the test set, we generate train and test split out of the available dataset. We split the original train dataset into training and validation subsets using a stratified random sample split by database id.

After removing question where evidence was not presented, we obtain in total 10163 question-evidence and question-SQL queries pairs. The obtained dataset statistics are provided in Table II.

TABLE II
DATASET FOR EXPERIMENTAL DESIGN

Dataset split	All records	Records with evidence	Nr. of DB
Train (train subset)	5657	5269	69
Train (validation subset)	3771	3514	69
Dev	1534	1380	11
Total	**10962**	**10163**	**80**

The 11 databases in Dev subset are different from the 69 databases in Train subsets.

We conduct 2 types of experiments:

- **A. Question-Evidence (Q-EVD):** This experiment involves retrieving the most relevant evidences within the evidence pool based on the embeddings of the questions.
- **B. Question-SQL Query (Q-SQL):** Similarly, this experiment involves matching questions to the most appropriate SQL queries based on their embeddings.

B. Fine-tuning

We utilize a pre-trained sentence transformer model, specifically the "all-MiniLM-L6-v2" which features a compact architecture with millions of parameters optimized for generating embeddings. This model is fine-tuned to adapt its capabilities more closely to the domain-specific requirements of SQL query generation from natural language queries.

1) Training Details: **Dataset**: The fine-tuning is conducted on a subset of the BIRD benchmark dataset, which includes 5,269 positive pairs. These pairs consist of corresponding natural language questions and their correct SQL queries, ensuring that the model learns the direct correlation between the query texts and the SQL commands. **Loss Function**: We employ MultipleNegativesRankingLoss with cosine similarity as the distance metric. This choice is aimed at enhancing the model's ability to differentiate between closely related texts (questions and SQL pairs), focusing on pulling positive samples closer in the embedding space while pushing unrelated (negative) samples farther away. **Hardware Utilization**: Training is performed on a T4 GPU complemented by 15 GB of RAM, which

allows for efficient handling of model computations and data loading. **Training Regime**: The model undergoes 10 epochs of training with 43 warm-up steps to ensure gradual learning rate adjustments, optimizing the training process for better convergence. **Evaluation Setup:** The models are evaluated under two distinct scenarios to assess their versatility and robustness:

In-database Evaluation: This involves testing the models on the same databases that were used during the fine-tuning process. It tests the model's performance in familiar contexts, using a hold-out subset from the train-validation partitions detailed in Chapter 3 of our documentation. **Out-of-database Evaluation**: This scenario tests the models on entirely new databases, which were not part of the training data. This is crucial for assessing the model's generalization capabilities across different data schema and SQL query structures. The datasets used here are from the development subset also described in Chapter 3. These evaluation setups help in understanding both the efficacy of the model in conditions similar to its training environment ("In-database") and its ability to adapt to new, previously unseen environments ("Out-of-database").

V. Empirical results

A. Initial Experiment

Using our evaluation method we report the initial results for selected models for experiment A (question-evidence) and B (question-SQL query). For open-source models we include the size of the models (in million parameters) and embedding dimensions of each model (Table III).

TABLE III
COMPARATIVE RESULTS

Model Name	Size	Emb. Dim.	MTEB bench.	Q-EVD	Q-SQL
all-mpnet-base-v2	110	768	43.81	68.13	64.46
all-MiniLM-L6-v2	23	384	41.95	69.63	74.66
OpenAI-3-small	-	1536	51.08	81.90	79.51
OpenAI-3-large	-	3072	**55.44**	**88.05**	79.46
OpenAI-ada-002	-	1536	49.25	82.54	**80.62**
Cohere-eng-v3	-	1024	55	80.09	68.18

Note: Metric: NDCG@10. Q-EVD: Experiment with question-evidence parirs. Q-SQL: Experiment with question-SQL query pairs.

For both experiments with SQL queries and questions the highest score is achieved by OpenAI embedding models. Additionally, a higher range score overall can be noticed in the text-to-sql evaluation seething, which tells about the difference in complexity of the tasks.

B. Fine-tuning

In this study, we evaluate the fine-tuned open-source sentence transformer model all-MiniLM-L6-v2, which is the smallest model among the ones chosen for the baseline. The model has been shown to generate meaningful and contextually rich sentence embeddings essential for dynamic information retrieval tasks [16]. The evaluation methodology was standardized across all models to ensure comparability of results.

Given the dataset's composition of exclusively positive pairs, the MultipleNegativesRankingLoss was employed for fine-tuning. This loss function is optimal for datasets without inherent negative examples, as it forces the model to differentiate between a positive instance and multiple negative instances within the same batch.

The effectiveness of the fine-tuned model is demonstrated in the comparative results table below.

TABLE IV
FINE-TUNED MODEL COMPARISON

Experiment	Question-Evidence		Question-SQL query	
Model Name	In-DB	Out-of-DB	In-DB	Out-of-DB
all-mpnet-base-v2	65.11	63.59	67.69	70.93
all-MiniLM-L6-v2	75.55	70.23	68.86	71.56
OpenAI-3-large	80.27	75.08	87.85	88.10
OpenAI-3-small	80.53	75.01	81.56	81.34
OpenAI-ada-002	81.62	76.53	82.00	83.17
Cohere-eng-v3	80.66	76.36	67.49	70.53
RAGSQL-text	**91.80**	**84.99**	84.50	83.07
RAGSQL-code	85.21	77.91	**93.02**	**89.62**

Note: Metric: NDCG@10. Scores in bold indicate the top performance in each category. In-DB - train validation subset. Out-of-DB represents dev subset.

The fine-tuned sentence transformer models outperforms all state-of-the-art models chosen for comparison across all evaluated metrics, indicating a high level of accuracy in retrieving the most relevant document. Furthermore, the model demonstrates robust performance in the both in-database and out-of-database setting.

VI. Conclusion

This paper addresses the role of RAG in enhancing text-to-SQL tasks, particularly within the context of LLMs tasked with generating SQL queries from natural language inputs. By implementing a custom evaluation framework and adapting data from the widely recognized BIRD bench, our study benchmarks existing embedding models and introduces a fine-tuned sentence transformer model that outperforms state-of-the-art models in both question-to-evidence and question-to-SQL retrieval experiments.

We suggest that the BIRD benchmark adaptation and the evaluation methodology presented in this paper can serve as a robust method for evaluating the quality of other embedding models focused on text-to-SQL tasks. Our methodological approach and the focus on both in-database and out-of-database evaluations demonstrate a comprehensive method of model testing and validation, ensuring that our findings are robust and applicable across various database environments.

The fine-tuned sentence transformer models generalize effectively across multiple professional domains. We demonstrate that much smaller models with efficient fine-tuning can lead to higher performance on a task. Having shown superior performance, the obtained models are not limited to our dataset but can be leveraged in various business real-world applications requiring text-to-SQL capabilities.

Our study demonstrates that fine-tuned sentence transformer models can generalize effectively across multiple domains, leading to significant performance improvements in text-to-SQL tasks. These findings align with recent research highlighting the potential of fine-tuned models in enhancing NLP tasks through robust domain adaptation strategies [18].

Together with results we share the data used for training and the obtained models.

ACKNOWLEDGMENT

This work was financially supported by the UNWE Research Programme.

REFERENCES

[1] N. Muennighoff, N. Tazi, L. Magne, and N. Reimers, "MTEB: Massive Text Embedding Benchmark," *arXiv preprint arXiv:2210.07316*, 2022. [Online]. Available: https://arxiv.org/abs/2210.07316

[2] T. B. Brown et al., "Language Models are Few-Shot Learners," *arXiv preprint arXiv:2005.14165*, 2020. [Online]. Available: https://arxiv.org/abs/2005.14165

[3] J. Wei, X. Wang, D. Schuurmans, M. Bosma, and E. Chi, "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," *arXiv preprint arXiv:2201.11903*, 2022. [Online]. Available: https://arxiv.org/abs/2201.11903

[4] J. Li, B. Hui, G. Qu, J. Yang, B. Li, B. Li, B. Wang, B. Qin, R. Geng, N. Huo, et al., "Can LLM Already Serve as a Database Interface? A Big Bench for Large-Scale Database Grounded Text-to-SQLs," *Advances in Neural Information Processing Systems*, vol. 36, 2024.

[5] C. Guo, Z. Tian, J. Tang, S. Li, Z. Wen, K. Wang, and T. Wang, "Retrieval-Augmented GPT-3.5-Based Text-to-SQL Framework with Sample-Aware Prompting and Dynamic Revision Chain," *arXiv preprint arXiv:2307.05074v2 [cs.IR]*, submitted on Jul. 11, 2023, last revised Sep. 4, 2023 (this version, v2).

[6] T. Yu, R. Zhang, K. Yang, M. Yasunaga, D. Wang, Z. Li, J. Ma, I. Li, Q. Yao, S. Roman, Z. Zhang, and D. Radev, "Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task," *arXiv preprint arXiv:1809.08887v5 [cs.CL]*, submitted on Sep. 24, 2018, last revised Feb. 2, 2019 (this version, v5), presented at EMNLP 2018. DOI: 10.48550/arXiv.1809.08887.

[7] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the Middle: How Language Models Use Long Contexts," *arXiv preprint arXiv:2307.03172*, 2023. [Online]. Available: https://arxiv.org/abs/2307.03172

[8] J. Zhang and others, "Domain Adaptation of a State of the Art Text-to-SQL Model: Lessons Learned and Challenges Found," *arXiv preprint arXiv:2312.05448*, 2023. [Online]. Available: https://arxiv.org/abs/2312.05448

[9] F. Wang and others, "Can LLM Already Serve as A Database Interface? A Big Bench for Large-Scale Database Grounded Text-to-SQLs," *arXiv preprint arXiv:2305.03111*, 2023. [Online]. Available: https://arxiv.org/abs/2305.03111

[10] P. Lewis and others, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *arXiv preprint arXiv:2005.11401*, 2020. [Online]. Available: https://arxiv.org/abs/2005.11401

[11] J. Smith and others, "Retrieval augmented text-to-SQL generation for epidemiological question answering using electronic health records," *arXiv preprint arXiv:2403.09226*, 2024. [Online]. Available: https://arxiv.org/abs/2403.09226

[12] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," *arXiv preprint arXiv:1908.10084*, 2019. [Online]. Available: https://arxiv.org/abs/1908.10084

[13] Xinyu Pi, Bing Wang, Yan Gao, Jiaqi Guo, Zhoujun Li, Jian-Guang Lou, "Towards Robustness of Text-to-SQL Models Against Natural and Realistic Adversarial Table Perturbation," *arXiv preprint arXiv:2212.09994*, 2022. [Online]. Available: https://arxiv.org/abs/2212.09994

[14] Shijie Chen, Ziru Chen, Huan Sun, Yu Su, "Error Detection for Text-to-SQL Semantic Parsing," *arXiv preprint arXiv:2305.13683v2*, 2023. [Online]. Available: https://arxiv.org/abs/2305.13683v2

[15] Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George van den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, Diego de Las Casas, Aurelia Guy, Jacob Menick, Roman Ring, Tom Hennigan, Saffron Huang, Loren Maggiore, Chris Jones, Albin Cassirer, Andy Brock, Michela Paganini, Geoffrey Irving, Oriol Vinyals, Simon Osindero, Karen Simonyan, Jack W. Rae, Erich Elsen, Laurent Sifreu, "Improving language models by retrieving from trillions of tokens," *arXiv preprint arXiv:2112.04426*, 2021. [Online]. Available: https://arxiv.org/abs/2112.04426

[16] Ziwei Ji, Zihan Liu, Nayeon Lee, Tiezheng Yu, Bryan Wilie, Min Zeng, Pascale Fung, "RHO (ρ): Reducing Hallucination in Open-domain Dialogues with Knowledge Grounding," *arXiv preprint arXiv:2212.01588*, 2022. [Online]. Available: https://arxiv.org/abs/2212.01588

[17] Zhi Chen, Lu Chen, Yanbin Zhao, Ruisheng Cao, Zihan Xu, Su Zhu, and Kai Yu. 2021. ShadowGNN: Graph Projection Neural Network for Text-to-SQL Parser. In Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, pages 5567–5577, Online. Association for Computational Linguistics.[Online]. Available: https://aclanthology.org/2021.naacl-main.441/

[18] Haoyang Li, Jing Zhang, Hanbing Liu, Ju Fan, Xiaokang Zhang, Jun Zhu, Renjie Wei, Hongyan Pan, Cuiping Li, Hong Chen, "CodeS: Towards Building Open-source Language Models for Text-to-SQL," *arXiv preprint arXiv:2402.16347v1*, 2024. [Online]. Available: https://arxiv.org/abs/2402.16347v1